

Cosmo J. Chou | Senior Software Engineer

cosmo.j.chou@gmail.com • (205) 561-9097 • Tuscaloosa, AL • Github: [anti0ch](#)

PROFESSIONAL SUMMARY

I'm a senior software engineer, most recently the recommendation-engine ML lead at a travel SaaS. In addition to that I owned the frontend, the backend, the AWS infrastructure, and a geospatial routing service alongside it. I also build multi-agent LLM tooling, including a review pipeline that tries to falsify its own findings before a human sees them, because I'm pretty neurotic about getting AI-in-the-loop right. Before software my background was mechanical engineering, three years of graduate research in instrumentation and real-time sensor fusion for transfemoral prosthetics. I'm in Alabama now and would love to relocate.

PROFICIENCIES

Languages: TypeScript (React, NestJS, Next.js), Python, PHP (Laravel, WordPress), SQL (Postgres / PostGIS / MySQL), HCL (Terraform).

Frontend: React 18, state management (Redux primary; also MobX, Zustand), webpack Module Federation, Vite, Cypress, Playwright, Tailwind, design system integration, A/B test instrumentation.

Mobile: React Native and Expo.

Infrastructure: AWS (Lambda, Fargate, Step Functions, S3, Elastic Beanstalk, DynamoDB, IAM), Cloudflare (Workers, D1, Wrangler), Terraform, CI/CD via GitHub Actions, Docker.

Observability / Release: Datadog (APM, RUM, log monitors, profiler), Artillery load testing, post-deploy benchmarking harnesses, incident-driven RCA.

LLM / AI Systems: Multi-agent adversarial review pipelines (Claude sub-agents + codex CLI in parallel), MCP server integration, agentic workflow design, falsification of model outputs.

Machine Learning: Recommendation systems, multi-factor retrieval/ranking pipelines, candidate staging for low-latency reads, model-drift monitoring, vector-embedding RAG (prototype-stage).

Algorithms / Correctness: Combinatorial optimization (max-coverage, greedy with swap local search, lazy evaluation scoped to where it is provably exact), explainable ranking, property-based and differential testing, evaluation harnesses scored against hand-corrected ground truth. Offline experiment design: paired comparisons against baselines and a clairvoyant ceiling, held-out parameter tuning, preregistered protocols.

PROFESSIONAL EXPERIENCE

RV LIFE | *Software Engineer* | October 2020 – May 2026

Frontend / full-stack

- **Built our map surface, then led it onto HERE Maps.** Started on Leaflet and extended it until it was effectively our own mapping layer, then led the migration onto HERE Maps when they shipped theirs. Also built the event-logging pipeline behind our A/B experiments, instrumenting the React side and wiring the data through for analysis.
- **Made an 86-spec frontend migration safe to run.** We were moving off a webpack Module Federation host onto a standalone Vite host. I catalogued every Cypress regression spec by what it actually depended on (61 real-backend, 21 hybrid, 4 mockable) so the porting plan was defensible, then built a Playwright harness against a real backend to cover the gap. Found a production stale-service-worker bug on the way.

Backend / platform / infra

- **Took myself off the rollback on-call.** Hit the manual Elastic Beanstalk rollback path during a production incident, then specced the one-click version so nobody had to be paged for it. Built the Terraform version-tagging underneath it. Ran across five non-prod environments and prod.
- **Ran production incidents end to end.** Same-day rollback-and-fix on a release regression I traced to a design-system portal bug on mobile; incident manager on a visitor-facing PII exposure, using git archaeology to bound the exposure window; live triage on a P1 platform outage, walking our API fleet down to zero and back while the upstream database host recovered, then filing the circuit-breaker work so it couldn't cascade again.
- **Found and fixed four security holes.** During my performance work I found four CVSS-Medium+ issues, fixed all four, and folded 15+ scattered PII compliance issues into a single shippable ticket against our web

monorepo.

- **Cut visitor-view latency on our itinerary API from 16s to under 1s.** I inherited the API after our Laravel-to-NestJS migration; it took the most traffic and was too slow to let us deprecate Laravel, so I untangled the external calls, fixed their caching, and made them async where feasible. Measured with a Python harness pairing real Datadog traffic against Artillery load runs (p50 down 27% on `/trips`).
- **Capacity-planned the legacy cutover.** Built the load model for moving classic Trip Wizard traffic onto the new API, with ramp-readiness criteria and rollback triggers. Measured the actual traffic mix to anchor it (the new backend was 20% of routing requests but 70% of response bytes, because it re-routed the whole trip on every edit) and filed the fixes that fell out, from a missing index on the auth-path query to capping fleet connections against a shared database.

Geospatial routing & HERE API integration

- **Gave routing custom no-go zones.** Swapped a brittle static HERE link-ID system for a polygon-based avoid-areas API on DynamoDB, so we could carve out constraints like bus-only roads, propane-restricted tunnels, and low-clearance overpasses.
- **Scored every road for drivability.** Built an engine that turns HERE ADAS data into curvature, slope, roughness (IRI), and speed-limit-variance scores, blended into one 0-10 number on every routing response. Easy roads skip the math by trusting HERE's own quality buckets as a fast path, so only the ambiguous ones get the expensive per-geometry evaluation.
- **Owned the HERE integration.** Ran the NestJS service in front of HERE's routing, matrix, isoline, and map-attributes APIs. It retries automatically in bus mode when a route hits a vehicle restriction, chunks and parallelizes calls to stay under the SMAP rate limit, and snaps to access coordinates instead of POIs so trucks land on the right roads.

ML / discovery (original mandate)

- **Built Pathfinder, our recommendation service.** Personalized next-stop suggestions over a 1M+ traveler dataset. Python on AWS Lambda and Fargate behind a GraphQL surface, with candidates precomputed to S3 through Step Functions so reads come back near-instantly.
- **Kept Pathfinder from breaking quietly.** Built the operational layer around it: CI/CD with release-please auto-deploys, circuit breakers with backoff on every Lambda and Fargate batch, structured logging and tracing, and a daily smoke-test suite whose OpenAPI contract checks catch schema drift before anyone notices.
- **Prototyped RAG for better recommendations.** Built a version of the destination recommender that embeds user-written campground reviews as vectors and blends that similarity with the structured scoring, to catch preference signal that rating, distance, and class miss on their own. Validated in dev, not yet shipped.

AI / agentic systems

- **Built a PR reviewer out of AI agents.** A custom Claude Code setup whose core piece runs Claude sub-agents and OpenAI's codex CLI in parallel against every diff and tries to falsify each finding before a human sees it. The same toolkit handles work-log automation, Jira hygiene, and MCP integrations across our internal platforms.

UNIVERSITY OF ALABAMA | *Graduate Research Assistant* | 2016 – 2019

- **Real-time sensor fusion for prosthetic legs.** Designed and built a wearable gait-measurement rig (encoders, IMUs, insole force sensors) and fused the on-body signal stream under real-time control. Co-authored, IEEE SoutheastCon 2019.

EDUCATION

B.S. Mechanical Engineering, University of Alabama, 2016